
Resumo de Prova Python

Testes com Pytest · Programação Orientada a Objetos · Decorators

Classes abstratas • Métodos estáticos e de classe • *args / **kwargs

Sumário

1. Testes Unitários com Pytest
2. Programação Orientada a Objetos (POO)
3. Decorators
4. Combinando os três temas
5. Erros comuns e pegadinhas de prova
6. Perguntas e respostas rápidas (revisão)
7. Checklist final

1. Testes Unitários com Pytest

1.1 Instalação e comandos essenciais

```
pip install pytest

pytest                # roda todos os testes do diretório
pytest test_arquivo.py  # roda um arquivo específico
pytest test_arquivo.py::test_funcao  # roda uma função específica
pytest test_arquivo.py::TestClasse::test_metodo  # roda um método de classe
pytest -v              # verboso: mostra nome de cada teste
pytest -q              # modo silencioso (quiet)
pytest -k "soma"       # roda testes cujo nome contém "soma"
pytest -x              # para na primeira falha
pytest --maxfail=2     # para após 2 falhas
pytest --tb=short       # traceback resumido
pytest -m marcador     # roda só testes com esse marker
pytest --collect-only  # só lista os testes, sem rodar
pytest -s              # mostra prints/saídas no console
pytest --cov=meu_pacote # cobertura de código (precisa pytest-cov)
```

1.2 Convenções de descoberta automática (auto-discovery)

O pytest encontra testes automaticamente seguindo estas regras — é um dos pontos mais cobrados em prova:

- Arquivos: `test_*.py` ou `*_test.py`
- Funções: precisam começar com `test_`
- Classes: precisam começar com `Test` e **não podem ter** `__init__`
- Métodos dentro da classe de teste: também precisam começar com `test_`

```
# test_calculadora.py
def test_soma():
    assert 2 + 2 == 4

class TestCalculadora:
    def test_subtracao(self):
        assert 5 - 3 == 2

    def test_multiplicacao(self):
        assert 3 * 3 == 9
```

1.3 Assertions

O pytest usa o `assert` nativo do Python (diferente do `unittest`, que usa `self.assertEqual()`). Internamente ele reescreve o `assert` (*assertion rewriting*) para mostrar mensagens de erro detalhadas quando falha.

```
def test_valores():
    assert soma(2, 3) == 5
    assert "abc" in "abcdef"
    assert isinstance(3, int)
    assert not False
    assert [1, 2, 3] == [1, 2, 3]
    assert 3.0001 == pytest.approx(3, rel=1e-2)  # comparação de floats
```

1.4 Testando exceções

```
import pytest

def divide(a, b):
    if b == 0:
        raise ValueError("Divisão por zero")
    return a / b

def test_divisao_por_zero():
    with pytest.raises(ValueError):
        divide(10, 0)

def test_mensagem_erro():
    with pytest.raises(ValueError, match="Divisão"):
        divide(10, 0)

def test_capturando_a_excecao():
    with pytest.raises(ValueError) as exc_info:
        divide(10, 0)
    assert "zero" in str(exc_info.value)
```

1.5 Fixtures

Fixtures preparam dados/recursos reutilizáveis para os testes — substituem o `setUp` / `tearDown` do `unittest`.

```
import pytest

@pytest.fixture
def lista_numeros():
    return [1, 2, 3, 4, 5]

def test_soma_lista(lista_numeros):
    assert sum(lista_numeros) == 15
```

Escopo (`scope`)

Escopo	Quando roda
<code>function</code> (padrão)	uma vez para cada função de teste
<code>class</code>	uma vez por classe de teste
<code>module</code>	uma vez por arquivo
<code>session</code>	uma vez para toda a execução do pytest

Setup/teardown com `yield`

```
@pytest.fixture
def conexao_bd():
    conexao = criar_conexao() # setup
    yield conexao             # entrega o valor ao teste
    conexao.fechar()           # teardown (roda depois do teste)

def test_query(conexao_bd):
    resultado = conexao_bd.executar("SELECT 1")
    assert resultado is not None
```

`conftest.py`

Fixtures definidas em um arquivo `conftest.py` ficam disponíveis automaticamente para todos os testes da pasta (e subpastas), **sem precisar de import**. É o local certo para fixtures compartilhadas.

Fixtures embutidas úteis

Fixture	Para quê serve
<code>tmp_path</code>	cria um diretório temporário único por teste

capsys	captura o que foi impresso com <code>print()</code>
monkeypatch	substitui/mocka atributos, variáveis de ambiente, funções

```
def test_print(capsys):
    print("olá")
    saida = capsys.readouterr()
    assert saida.out == "olá\n"

def test_monkeypatch(monkeypatch):
    monkeypatch.setenv("MOD0", "teste")
    assert os.environ["MOD0"] == "teste"
```

1.6 Parametrização — `@pytest.mark.parametrize`

Roda o mesmo teste várias vezes com valores diferentes, evitando repetir código.

```
@pytest.mark.parametrize("a, b, esperado", [
    (2, 3, 5),
    (0, 0, 0),
    (-1, 1, 0),
])
def test_soma_parametrizada(a, b, esperado):
    assert soma(a, b) == esperado
```

Pode combinar múltiplos `@parametrize` empilhados: o pytest gera o produto cartesiano dos valores (todas as combinações).

1.7 Marks (marcadores)

```
@pytest.mark.skip(reason="ainda não implementado")
def test_futuro():
    ...

@pytest.mark.skipif(sys.platform == "win32", reason="só roda em Linux")
def test_condicional():
    ...

@pytest.mark.xfail(reason="bug conhecido, aguardando correção")
def test_com_bug():
    ...

@pytest.mark.slow
def test_demorado():
    ...
```

1.8 Estrutura típica de projeto

```
meu_projeto/
├── src/
│   └── calculadora.py
├── tests/
│   ├── conftest.py
│   └── test_calculadora.py
└── pytest.ini (ou pyproject.toml)
```

1.9 Dicas rápidas para a prova

- `assert` simples, e não `assertEqual` (isso é do `unittest`).
- Fixtures usam `yield` para separar setup de teardown.
- `pytest.raises()` é a forma padrão de testar exceções.
- `parametrize` evita duplicação de testes.
- Nome do arquivo/função **sempre** precisa começar com `test_` para o auto-discovery funcionar.
- `conftest.py` compartilha fixtures sem precisar importar.

2. Programação Orientada a Objetos (POO)

2.1 Classes Abstratas

Definem um "contrato": uma classe base que não pode ser instanciada diretamente, forçando as subclasses a implementar certos métodos.

```
from abc import ABC, abstractmethod

class Forma(ABC):
    # herda de ABC
    @abstractmethod
    def area(self):
        # método abstrato: sem implementação
        pass

    @abstractmethod
    def perimetro(self):
        pass

    def descricao(self):
        # método concreto normal, pode conviver
        return "Sou uma forma geométrica"

class Retangulo(Forma):
    def __init__(self, largura, altura):
        self.largura = largura
        self.altura = altura

    def area(self):
        return self.largura * self.altura

    def perimetro(self):
        return 2 * (self.largura + self.altura)
```

Pegadinha clássica: `Forma()` gera `TypeError: Can't instantiate abstract class Forma with abstract methods area, perimetro`. Já `Retangulo(3, 4)` funciona normalmente, porque implementou **todos** os métodos abstratos da classe pai. Se a subclasse esquecer de implementar até um só método abstrato, ela também não pode ser instanciada.

2.2 Métodos estáticos, de classe e de instância

```
class Pessoa:
    populacao = 0

    def __init__(self, nome):
        self.nome = nome
        Pessoa.populacao += 1

    def apresentar(self):
        # método de instância
        return f"Eu sou {self.nome}"

    @classmethod
    def total_pessoas(cls):
        # recebe a classe (cls)
        return cls.populacao

    @classmethod
    def criar_anonimo(cls):
        # "factory method" comum de prova
        return cls("Anônimo")

    @staticmethod
    def eh_maior_idade(idade):
        # não recebe self nem cls
        return idade >= 18
```

Tipo	Recebe	Uso típico
método normal	<code>self</code>	acessa/altera atributos da instância
<code>@classmethod</code>	<code>cls</code>	acessa/altera atributos da classe; "factory methods"
<code>@staticmethod</code>	nada	função utilitária relacionada ao tema da classe

2.3 Herança, `super()` e polimorfismo

```
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def emitir_som(self):
        raise NotImplementedError

class Cachorro(Animal):
    def __init__(self, nome, raca):
        super().__init__(nome) # chama __init__ da classe pai
        self.raca = raca

    def emitir_som(self):
        return "Au au!"

class Gato(Animal):
    def emitir_som(self):
        return "Miau!"

animais = [Cachorro("Rex", "Labrador"), Gato("Mimi")]
for a in animais:
    print(a.emitir_som()) # polimorfismo: mesma chamada, comportamentos diferentes
```

2.4 Encapsulamento

Convenção	Significado
<code>publico</code>	acesso livre
<code>_protegido</code>	convenção: "uso interno", mas ainda acessível
<code>__privado</code>	<i>name mangling</i> : Python renomeia para <code>_Classe__privado</code>

2.5 Properties (getter/setter "pythônico")

```
class Circulo:
    def __init__(self, raio):
        self._raio = raio

    @property
    def raio(self):
        return self._raio

    @raio.setter
    def raio(self, valor):
        if valor <= 0:
            raise ValueError("Raio deve ser positivo")
        self._raio = valor

    @property
    def area(self):
        return 3.14159 * self._raio ** 2

c = Circulo(2)
print(c.area) # chamado sem parênteses, como atributo
c.raio = 5    # passa pelo setter e valida
```

2.6 Métodos mágicos (*dunder methods*) mais cobrados

Método	Quando é chamado
<code>__init__</code>	ao criar o objeto
<code>__str__</code>	<code>print(obj)</code> / <code>str(obj)</code>
<code>__repr__</code>	representação "oficial" (debug, console)
<code>__eq__</code>	<code>obj1 == obj2</code>

<code>__len__</code>	<code>len(obj)</code>
<code>__add__</code>	<code>obj1 + obj2</code>

3. Decorators

3.1 O que é um decorator

É uma função que recebe outra função e retorna uma versão "modificada" dela, sem alterar o código original. Usa o `@` como açúcar sintático.

```
def meu_decorator(func):
    def wrapper(*args, **kwargs):
        print("Antes da execução")
        resultado = func(*args, **kwargs)
        print("Depois da execução")
        return resultado
    return wrapper

@meu_decorator
def saudacao(nome):
    print(f"Olá, {nome}!")

saudacao("Gabriel")
# Antes da execução
# Olá, Gabriel!
# Depois da execução
```

`@meu_decorator` acima da função é equivalente a escrever:

```
saudacao = meu_decorator(saudacao)
```

3.2 Por que usar `*args` e `**kwargs` no wrapper

O `wrapper` precisa aceitar **qualquer** assinatura de função a ser decorada — senão o decorator só funcionaria em funções com uma quantidade fixa e conhecida de parâmetros.

- `*args` → captura argumentos posicionais numa **tupla**
- `**kwargs` → captura argumentos nomeados num **dicionário**

```
def log(func):
    def wrapper(*args, **kwargs):
        print(f"Chamando {func.__name__} com args={args}, kwargs={kwargs}")
        return func(*args, **kwargs)
    return wrapper

@log
def soma(a, b, c=0):
    return a + b + c

soma(1, 2, c=3)
# Chamando soma com args=(1, 2), kwargs={'c': 3}
```

Atenção: `*args` e `**kwargs` são apenas convenções de nome — o que importa são os símbolos `*` e `**`. O Python também permite `*a`, `**kw`, por exemplo. Na chamada da função, esses mesmos símbolos servem para "desempacotar" uma lista/dict existente:

```
valores = (1, 2)
extras = {"c": 3}
soma(*valores, **extras) # equivale a soma(1, 2, c=3)
```

3.3 Decorator com parâmetros próprios

Precisa de **três** camadas de função: a externa recebe os parâmetros do decorator, a do meio recebe a função, e a interna (`wrapper`) recebe os argumentos da chamada.

```
def repetir(vazes):
    def decorator(func):
        def wrapper(*args, **kwargs):
            resultado = None
            for _ in range(vazes):
                resultado = func(*args, **kwargs)
            return resultado
        return wrapper
    return decorator

@repetir(vazes=3)
def oi():
    print("oi!")

oi()
# oi!
# oi!
# oi!
```

3.4 `functools.wraps`

Sem esse detalhe, a função decorada "perde" seu nome e docstring originais (passam a ser os do `wrapper`). `@wraps(func)` preserva essas metainformações — item clássico de prova.

```
from functools import wraps

def meu_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@meu_decorator
def exemplo():
    """Docstring original"""
    pass

print(exemplo.__name__) # 'exemplo' (e não 'wrapper')
print(exemplo.__doc__) # 'Docstring original'
```

3.5 Empilhando múltiplos decorators

São aplicados de baixo para cima (o mais próximo da função roda primeiro).

```
@decorator_A
@decorator_B
def minha_funcao():
    ...

# equivale a: minha_funcao = decorator_A(decorator_B(minha_funcao))
```

3.6 Decorators embutidos usados com classes

```
class Calculadora:
    @staticmethod
    def dobro(x):
        return x * 2

    @classmethod
    def criar_padrao(cls):
        return cls()

    @property
    def resultado(self):
        return self._resultado
```

3.7 `functools.lru_cache` (bônus comum)

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
# cacheia resultados já calculados, evitando recomputar
```

4. Combinando os três temas

Pegadinha comum de prova: testar com pytest uma classe abstrata cujo método concreto usa decorator.

```
from abc import ABC, abstractmethod
import pytest
from functools import wraps

def validar_positivo(func):
    @wraps(func)
    def wrapper(self, *args, **kwargs):
        if any(v < 0 for v in args):
            raise ValueError("Valores não podem ser negativos")
        return func(self, *args, **kwargs)
    return wrapper

class Forma(ABC):
    @abstractmethod
    def area(self):
        pass

class Quadrado(Forma):
    def __init__(self, lado):
        self.lado = lado

    @validar_positivo
    def area(self, extra=0):
        return self.lado ** 2 + extra

def test_area_quadrado():
    q = Quadrado(4)
    assert q.area() == 16

def test_area_negativa_gera_erro():
    q = Quadrado(4)
    with pytest.raises(ValueError):
        q.area(-1)

def test_forma_nao_instanciavel():
    with pytest.raises(TypeError):
        Forma()
```

5. Erros comuns e pegadinhas de prova

1. Esquecer `@wraps(func)` no wrapper → `func.__name__` vira `"wrapper"` em qualquer função decorada.
2. Tentar instanciar uma classe abstrata (ou uma subclasse que não implementou todos os métodos abstratos) → `TypeError`.
3. Confundir `@staticmethod` com `@classmethod`: se a questão usa `cls` no corpo do método, é `@classmethod`; se não usa nem `self` nem `cls`, é `@staticmethod`.
4. Achar que decorator com parâmetros precisa de só duas funções aninhadas — na verdade são **três** níveis (parâmetro do decorator → função → wrapper).
5. Nomear função/arquivo de teste sem o prefixo `test_` → o pytest simplesmente não descobre e não executa o teste (nenhum erro é mostrado, ele só "some").
6. Usar `assertEqual` em vez de `assert` (mistura de sintaxe do `unittest` com `pytest`).

6. Perguntas e respostas rápidas (revisão)

P: O que acontece se eu tentar instanciar uma classe com um `@abstractmethod` não implementado?

R: Python lança `TypeError` no momento da instanciação, informando quais métodos abstratos faltam implementar.

P: Qual a diferença entre `@staticmethod` e `@classmethod`?

R: `@staticmethod` não recebe nenhum parâmetro implícito (nem `self`, nem `cls`); `@classmethod` recebe `cls`, a própria classe, permitindo acessar/alterar atributos de classe ou criar instâncias alternativas.

P: Para que serve `*args` e `**kwargs` dentro de um wrapper de decorator?

R: Para permitir que o decorator funcione em qualquer função, independentemente da quantidade e tipo de argumentos que ela recebe.

P: Como o pytest identifica quais funções são testes?

R: Pelo nome: arquivos `test_*.py` / `*_test.py`, funções `test_*` e classes `Test*` (sem `__init__`).

P: Qual a vantagem de `@pytest.mark.parametrize` sobre escrever vários testes separados?

R: Evita duplicação de código: o mesmo corpo de teste roda automaticamente para cada conjunto de valores fornecido.

P: O que uma fixture com `yield` permite fazer que um `return` simples não permite?

R: Executar código de "limpeza" (teardown) depois que o teste terminar, como fechar conexões ou remover arquivos temporários.

7. Checklist final antes da prova

- ☐ Sei diferenciar `@staticmethod` de `@classmethod` (parâmetros que recebem)
- ☐ Sei explicar por que `ABC` + `@abstractmethod` impede instanciação
- ☐ Sei escrever um decorator básico com `*args, **kwargs`
- ☐ Sei escrever um decorator com parâmetros (3 níveis de função)
- ☐ Sei para que serve `functools.wraps`
- ☐ Sei empilhar múltiplos decorators e explicar a ordem de execução
- ☐ Sei escrever um teste pytest simples (`assert`)
- ☐ Sei usar `pytest.raises()` para testar exceções
- ☐ Sei criar e usar uma fixture com `yield` (setup/teardown)
- ☐ Sei usar `@pytest.mark.parametrize`
- ☐ Sei explicar o que é `conftest.py`
- ☐ Sei usar `@property` para criar getters/setters